

IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

Bài 11: Lập trình xử lý ngắt

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 1:

Chương trình:

.eqv IN_ADDRESS_HEX_KEYBOARD 0xFFFF0012 # Địa chỉ ghi hàng để quét phím

.eqv OUT_ADDRESS_HEX_KEYBOARD 0xFFFF0014 # Địa chỉ đọc giá trị phím được nhấn

.data

space: .asciz "\n"

.text

main:

li t1, IN_ADDRESS_HEX_KEYBOARD # (1) Gán địa chỉ IN

li t2, OUT_ADDRESS_HEX_KEYBOARD # (2) Gán địa chỉ OUT

polling:

li s0, 0x01 # (3) Bắt đầu từ hàng đầu tiên

li t4, 0x10 # (4) Giới hạn hàng là 0x10

scan_loop:

sb s0, 0(t1) # (5) Ghi hàng vào IN để quét

lb a0, 0(t2) # (6) Đọc mã phím từ OUT

beq a0, zero, next_row # (7) Không có phím → chuyển hàng

li a7, 34 # (8) In mã phím dạng hex

ecall

li a7, 4

la a0, space

ecall

(9) Ngủ 100ms

(10) Sang hàng tiếp theo

(11) Nếu chưa hết hàng $\rightarrow$ lặp lại

(12) Quét lại từ đầu

- Trong ứng mã **0x22**
 r s0 = 0x01 (hàng đầu tiên).

Kết quả:

No.	Vị trí	s0	t1	t2	t4	a0	Chú thích
1	(1)	0x00	0xffff0012	0x00000000	0x00	0x00	Gán địa chỉ ghi hàng vào t1
2	(2)	0x00	0xffff0012	0xffff0014	0x00	0x00	Gán địa chỉ đọc phím vào t2
3	(3)	0x01	0xffff0012	0xffff0014	0x00	0x00	Khởi tạo s0 = 0x01, bắt đầu quét hàng đầu
4	(4)	0x01	0xffff0012	0xffff0014	0x10	0x00	
5	(5)	0x01	0xffff0012	0xffff0014	0x10	0x00	Ghi s0 vào IN để quét hàng đầu tiên
6	(6)	0x01	0xffff0012	0xffff0014	0x10	0x00	Đọc giá trị OUT vào a0 (0x00)
7	(7)	0x01	0xffff0012	0xffff0014	0x10	0x00	OUT = 0, nhảy đến next(10)
8	(10)	0x02	0xffff0012	0xffff0014	0x10	0x00	Tăng hàng lên 1
9	(11)	0x02	0xffff0012	0xffff0014	0x10	0x00	Vẫn chưa vượt quá số hàng, tiếp tục quét hàng 2
10	(5)	0x02	0xffff0012	0xffff0014	0x10	0x00	Ghi s0 vào IN, quét hàng 2 (có phím 5 được nhấn)
11	(6)	0x02	0xffff0012	0xffff0014	0x10	0x22	Đọc giá trị OUT vào a0 (0x22)
12	(8)	0x02	0xffff0012	0xffff0014	0x10	0x22	In ra giá trị OUT ra màn hình

Mô tả thuật toán:

- Cần phải quét toàn bộ bàn phím 16 phím theo từng hàng.
- Bắt đầu từ hàng 1, khi quét xong 1 hàng, nếu giá trị trả về của phép kiểm tra a0 là
 - + Khác 0, tức là dòng này có phím nào đó được nhấn, sau đó ta in ra giá trị a0 là mã phím của phím này. Sau đó in ra và quay về quét bàn phím từ hàng 1
 - + Bằng 0, tức là dòng này không có phím nào được nhấn, ta chuyển sang dòng tiếp theo (hàm next_row), nếu cả 16 phím được kiểm tra mà giá trị a0 = 0, thì ta quay về hàng 1 để quét lại đến khi đợi được phím được nhấn.
- Chèn thêm ký tự xuống dòng giữa các lần in để nhìn rõ ràng hơn.

Assignment 2:

Chương trình:

```
.eqv IN_ADDRESS_HEX KEYBOARD 0xFFFF0012
```

```
.data
```

```
message: .asciz "Someone's pressed a button.\n"
```

```
# -----
```

```

# MAIN Procedure

# -----

.text

main:

# Load the interrupt service routine address to the UTVEC register
la t0, handler

csrrs zero, utvec, t0 #(1)

# Set the UEIE (User External Interrupt Enable) bit in UIE register
li t1, 0x100

csrrs zero, uie, t1 # uie - ueie bit (bit 8) #(2)

# Set the UIE (User Interrupt Enable) bit in USTATUS register
csrrsi zero, ustatus, 1 # ustatus - enable uie (bit 0)#(3)

# Enable the interrupt of keypad of Digital Lab Sim
li t1, IN_ADDRESS_HEXKEYBOARD

li t3, 0x80 # bit 7 = 1 to enable interrupt

sb t3, 0(t1) #(4)

# -----

# No-end loop, main program, to demo the effective of interrupt

# -----

loop: # đoạn code này giữ cho cpu chạy
nop
nop
nop
nop
j loop #(5)

end_main:

```

```
# -----  
# Interrupt service routine  
# -----  
handler: # khi có 1 phím được bấm thì cpu tự động nhảy vào đây để chạy  
# ebreak # Can pause the execution to observe registers  
# Saves the context  
addi sp, sp, -8  
sw a0, 0(sp)  
sw a7, 4(sp)  
# Handles the interrupt  
# Shows message in Run I/O  
li a7, 4  
la a0, message  
ecall  
# Restores the context  
lw a7, 4(sp)  
lw a0, 0(sp)  
addi sp, sp, 8  
# Back to the main procedure  
uret #(6)
```

Kết quả:

Vị trí	t0	utvec	t1	uie	ustatus	t3	pc	Ghi chú
1	0x00400038	0x00400038	0x00000000	0x00000000	0x00000000	0x00000000	0x00400000 đến 0x00400008	Tải địa chỉ chương trình ngắt vào thanh ghi t0
2	0x00400038	0x00400038	0x00000100	0x00000100	0x00000000	0x00000000	0x0040000c 0x00400010	Đặt bit ueie vào thanh ghi uie
3	0x00400038	0x00400038	0x00000100	0x00000100	0x00000001	0x00000000	0x00400014	Đặt bit uie trong thanh ghi ustatus
4	0x00400038	0x00400038	0xffff0012	0x00000100	0x00000001	0x00000080	0x00400018 Đến 0x00400024	Bật ngắt của Digital Lab Sim
5	0x00400038	0x00400038	0xffff0012	0x00000100	0x00000001	0x00000080	0x00400028 Đến 0x00400034	Nếu không có nút nào được ấn thì sẽ lặp vô hạn
6, 5	0x00400038	0x00400038	0xffff0012	0x00000100	0x00000001	0x00000080	0x00400038 Đến 0x00400060 Rồi quay lại 0x00400028	Khi có nút được ấn, nhảy đến handler cất a0, a7 vào stack, in message, rồi lấy lại a0, a7 và trở lại chương trình

Mô tả thuật toán:

- Chương trình này kích hoạt ngắt ngoài từ bàn phím hexa và xử lý ngắt bằng cách hiển thị thông báo khi có bất kỳ phím nào được nhấn, sử dụng cơ chế ngắt người dùng (user interrupt) trên hệ thống giả lập Digital Lab Sim.
- (1) Gán địa chỉ của trình phục vụ ngắt (interrupt service routine) vào thanh ghi utvec để CPU biết nhảy đến đâu khi có ngắt.
- (2) Bật bit UEIE trong thanh ghi uie để cho phép nhận ngắt ngoài từ người dùng (User External Interrupt Enable).
- (3) Bật bit UIE trong thanh ghi ustatus để kích hoạt xử lý ngắt người dùng.
- (4) Ghi 0x80 (bit 7 = 1) vào địa chỉ bàn phím 0xFFFF0012 để bật tính năng phát sinh ngắt khi có phím được nhấn.
- (5) Vào vòng lặp vô hạn (loop) thực thi các lệnh rỗng (nop). Chương trình chính không làm gì ngoài chờ ngắt xảy ra. Mỗi khi có phím được nhấn, CPU tự động nhảy đến trình phục vụ ngắt (handler).
- Trình phục vụ ngắt (handler) hoạt động như sau:
 - a. Lưu trạng thái trước khi xử lý: Lưu các thanh ghi a0 và a7 vào stack để không mất dữ liệu khi thực hiện xử lý ngắt.
 - b. Xử lý ngắt: Dùng syscall ecall để in chuỗi thông báo "Someone's pressed a button." ra cửa sổ Run I/O (hoặc console).
 - c. Khôi phục trạng thái: Nạp lại các giá trị thanh ghi a7, a0 từ stack.
 - d. Quay lại chương trình chính: Sử dụng lệnh uret để thoát khỏi trình phục vụ ngắt và quay về đúng nơi chương trình chính đang chạy dở.

Assignment 3:

```
.eqv IN_ADDRESS_HEX_KEYBOARD    0xFFFF0012  
.eqv OUT_ADDRESS_HEX_KEYBOARD    0xFFFF0014
```



```
.data
```

```
message: .asciz "Key scan code: "
```

```
# -----
```

```
# MAIN Procedure
```

```
# -----
```

```
.text
```

```
main:
```

```
    # Load the interrupt service routine address to the UTVEC register
```

```
    la    t0, handler
```

```
    csrrs zero, utvec, t0 #(1)
```

```
    # Set the UEIE (User External Interrupt Enable) bit in UIE register
```

```
    li    t1, 0x100
```

```
    csrrs zero, uie, t1    #(2) uie - ueie bit (bit 8)
```

```
    # Set the UIE (User Interrupt Enable) bit in USTATUS register
```

```
    csrrsi zero, ustatus, 1    #(3) ustatus - enable uie (bit 0)
```

```
    # Enable the interrupt of keypad of Digital Lab Sim
```

```
    li    t1, IN_ADDRESS_HEX_KEYBOARD
```

```
    li    t3, 0x80          #(4) bit 7 = 1 to enable interrupt
```

```
    sb    t3, 0(t1)
```

```
# -----
```

```
# Loop to print a sequence of numbers
```

```
# -----
```

```
xor    s0, s0, s0    #(5) s0 = 0 (count)
```

```
loop:
```

```
addi   s0, s0, 1     #(6) count = count + 1
```

```
prn_seq:
```

```
li     a7, 1
```

```
mv     a0, s0        #(7) Print current count
```

```
ecall
```

```
li     a7, 11
```

```
li     a0, '\n'      # Print newline
```

```
ecall
```

```
sleep:
```

```
li     a7, 32
```

```
li     a0, 300       # (8) Sleep 300 ms
```

```
ecall
```

```
j      loop
```

```
# -----
```

```
# Interrupt service routine
```

```
# -----
```

handler:

Save context

addi sp, sp, -16

sw a0, 0(sp)

sw a7, 4(sp)

sw t1, 8(sp)

sw t2, 12(sp)

Print message

li a7, 4

la a0, message

ecall

Quét từng hàng để tìm phím được nhấn

li t3, 0 # (9) t3: chỉ số hàng 0-3

scan_rows:

li t1, IN_ADDRESS_HEX_KEYBOARD

li t2, 0x80 # (10) bit 7 = 1 để bật ngắt của bàn phím

li t4, 1

sll t4, t4, t3 # (11) $t4 = t4 * 2^{t3}$

or t2, t2, t4 # (12) bật bit tương ứng với hàng cần quét

sb t2, 0(t1) # (13) Lưu vào địa chỉ bàn phím để quét

Đọc mã phím từ output

li t1, OUT_ADDRESS_HEXA_KEYBOARD

lb a0, 0(t1)

beq a0, zero, next_row #(14) Nếu không có phím được nhấn → tiếp tục hàng sau

In mã phím (hex)

li a7, 34 #(15)

ecall

li a7, 11

li a0, '\n' # In xuống dòng

ecall

next_row:

addi t3, t3, 1 #(16)

li t5, 4

blt t3, t5, scan_rows # (17) Nếu quét đến hàng cuối thì quét lại từ hàng 1

Khôi phục ngữ cảnh

lw t2, 12(sp)

lw t1, 8(sp)

lw a7, 4(sp)

lw a0, 0(sp)

addi sp, sp, 16 #(18)

uret

Kết quả:

Vị trí	t0	utvec	uie	ustatus	t1	t3	t2	a0	s0	PC
(1)	0x00400058	0x00400058	0x000	0x00	0x000	0	0x00	0x00000000		0x00400008
(2)	0x00400058	0x00400058	0x100	0x00	0x100	0	0x00	0x00000000		0x00400010
(3)	0x00400058	0x00400058	0x100	0x01	0x100	0	0x00	0x00000000		0x00400014
(4)	0x00400058	0x00400058	0x100	0x01	0xffff0012	0	0x00	0x00000000		0x00400020
(5)	0x00400058	0x00400058	0x100	0x01	0xffff0012	0	0x00	0x00000000	0	0x00400028
(6)	0x00400058	0x00400058	0x100	0x01	0xffff0012	0	0x00	0x00000000	1,2,..	0x0040002c
(8)	0x00400058	0x00400058	0x100	0x01	0xffff0012	0	0x00	0x00000000		0x0040004c
(9)	0x00400058	0x00400058	0x100	0x10	0xffff0012	0	0x00	-		0x0040007c
(10)	0x00400058	0x00400058	0x100	0x10	0xffff0012	0	0x80	-		0x00400088
(12)	0x00400058	0x00400058	0x100	0x10	0xffff0012	0	0x80	-		0x00400094
(13)	0x00400058	0x00400058	0x100	0x10	0xffff0012	0	0x81	-		0x00400098
(14)	0x00400058	0x00400058	0x100	0x10	0xffff0014	0	0x81	-		0x004000a8
(16)	0x00400058	0x00400058	0x100	0x10	0xffff0014	1	0x81	-		0x004000c0
(17)	0x00400058	0x00400058	0x100	0x10	0xffff0014	1	0x81	-		0x004000c8
(10)	0x00400058	0x00400058	0x100	0x10	0xffff0012	1	0x80	-		0x00400088
(12)	0x00400058	0x00400058	0x100	0x10	0xffff0012	1	0x80	-		0x00400094
(13)	0x00400058	0x00400058	0x100	0x10	0xffff0012	1	0x82	-		0x00400098
(14)	0x00400058	0x00400058	0x100	0x10	0xffff0014	1	0x82	0x00000042		0x004000ac
(15)	0x00400058	0x00400058	0x100	0x10	0xffff0014	1	0x82	-		0x004000c0
(16)	0x00400058	0x00400058	0x100	0x10	0xffff0014	2	0x82	-		0x004000c8
(17)	0x00400058	0x00400058	0x100	0x10	0xffff0014	2	0x82	-		0x00400088
(10)	0x00400058	0x00400058	0x100	0x10	0xffff0012	2	0x80	-		0x00400094
(12)	0x00400058	0x00400058	0x100	0x10	0xffff0012	2	0x80	-		0x00400098
(13)	0x00400058	0x00400058	0x100	0x10	0xffff0012	2	0x84	-		0x004000a8
(14)	0x00400058	0x00400058	0x100	0x10	0xffff0014	2	0x84	-		0x004000c0
(16)	0x00400058	0x00400058	0x100	0x10	0xffff0014	3	0x84	-		0x004000c8
(17)	0x00400058	0x00400058	0x100	0x10	0xffff0014	3	0x84	-		0x00400088
(10)	0x00400058	0x00400058	0x100	0x10	0xffff0014	3	0x80	-		0x00400094
(12)	0x00400058	0x00400058	0x100	0x10	0xffff0012	3	0x80	-		0x00400098
(13)	0x00400058	0x00400058	0x100	0x10	0xffff0012	3	0x88	-		0x004000a8
(14)	0x00400058	0x00400058	0x100	0x10	0xffff0014	3	0x88	-		0x004000c0
(16)	0x00400058	0x00400058	0x100	0x10	0xffff0014	4	0x88	-		0x004000c8
(18)	0x00400058	0x00400058	0x100	0x10	0xffff0012	4	0x00	0x00000000		0x004000dc

Mô tả:

Thuật toán chính:

- **Thiết lập ngắt:** cấu hình CSR và bật ngắt bàn phím.
- **Main loop:** in dãy số không ngừng, để chứng tỏ ISR độc lập.

- **ISR:**

1. Lưu ngữ cảnh.
2. In tiêu đề.
3. Quét 4 hàng của bàn phím, đọc scan-code, in nếu phát hiện.
4. Khôi phục ngữ cảnh, uret quay về main.

Cơ chế interrupt đảm bảo **phản hồi nhanh** với bàn phím mà không làm gián đoạn luồng chính in đếm.

Chi tiết:

1. **(1) Cấu hình vector ngắt**

- la t0, handler tải địa chỉ của hàm xử lý ngắt (handler) vào thanh ghi t0.
- csrrs zero, utvec, t0 ghi giá trị t0 vào CSR utvec, xác định địa chỉ nhảy tới khi ngắt xảy ra.

2. **(2) Bật UEIE (User External Interrupt Enable)**

- li t1, 0x100 đặt t1 = 0x100 (bit 8 = 1).
- csrrs zero, uie, t1 thiết lập bit 8 của CSR uie, cho phép ngắt ngoại vi ở chế độ User.

3. **(3) Bật UIE (User Interrupt Enable)**

- csrrsi zero, ustatus, 1 đặt bit 0 của CSR ustatus, cho phép xử lý ngắt người dùng (global interrupt enable).

4. **(4) Kích hoạt ngắt từ bàn phím**

- li t1, IN_ADDRESS_HEX_KEYBOARD lấy địa chỉ cổng quét bàn phím.
- li t3, 0x80 đặt t3 = 0x80 (bit 7 = 1).
- sb t3, 0(t1) ghi vào cổng quét, bật tính năng phát sinh ngắt khi có phím nhấn.

5. **(5) Khởi tạo biến đếm**

- xor s0, s0, s0 gán $s0 = 0$, làm biến đếm.

6. (6) Tăng biến đếm

- addi s0, s0, 1 mỗi vòng lặp tăng s0 thêm 1.

7. (7) In biến đếm

- li a7, 1 chuẩn bị syscall in số nguyên.
- mv a0, s0 đặt tham số.
- ecall gọi in.
- Tiếp theo in ký tự newline với syscall 11.

8. (8) Sleep 300 ms

- li a7, 32; li a0, 300; ecall dừng chương trình 300 ms trước khi quay lại lặp.

9. (11) Quay lại vòng lặp

- j loop nhảy về nhãn loop để tiếp tục in đếm, chờ ngắt xảy ra bất cứ lúc nào.

handler

10.(12) Lưu ngữ cảnh

- addi sp, sp, -16 cấp phát stack.
- sw a0,0(sp); sw a7,4(sp); sw t1,8(sp); sw t2,12(sp) lưu các thanh ghi a0,a7,t1,t2 lên stack.

11.(13) In thông báo

- li a7,4; la a0,message; ecall in chuỗi "Key scan code: ".

12.(9) Chuẩn bị quét hàng

- li t3,0 đặt $t3 = 0$, biến index hàng (0–3).

13.Scan từng hàng (scan_rows)

- (10) li t1, IN_ADDRESS_HEX_KEYBOARD; li t2,0x80 lấy cổng quét, khởi t2 với bit enable.
- (11) li t4,1; sll t4,t4,t3 tính $t4 = 1 \ll t3$, chọn hàng.

- (12) or t2,t2,t4 kết hợp bit enable và bit hàng.
- (13) sb t2,0(t1) ghi để quét hàng t3.
- Đọc scan-code: li t1, OUT_ADDRESS_HEX_KEYBOARD; lb a0,0(t1).
- (14) beq a0,zero,next_row nếu a0==0 (chưa có nhấn), nhảy sang hàng kế.

14.(15) In mã phím

- li a7,34; ecall in scan-code ở định dạng hex.
- In newline tương tự.

15.(16–17) Tiếp hàng

- addi t3,t3,1 tăng index hàng.
- li t5,4; blt t3,t5,scan_rows nếu t3 < 4, quét tiếp; ngược lại kết thúc vòng quét.

16.(18) Khôi phục ngữ cảnh và trả về

- lw t2,12(sp); lw t1,8(sp); lw a7,4(sp); lw a0,0(sp); addi sp,sp,16 lấy lại các thanh ghi.
- uret trả về địa chỉ lưu trong mepc, tiếp tục chương trình chính tại chỗ đã bị ngắt.